

Phonebook Lookup — Group Report (Parts A, B, D)

Group: G4 — Searching & Hash tables (§2.2) **Members:** **Topic:** Linear · Binary · Interpolation search · hashing & collisions **Demo:** Phone-book lookup with a hash table ($n \geq 100\,000$)

Written form of Parts A, B and D of the assignment
(C0B_Group_Project_Guide.pdf). Build/run instructions and the machine
used are in [../README.md](#).

Part A — Position & purpose

A.1 Syllabus map

Place each structure on the syllabus outline: chapter, section, page, and which learning outcome (LO1 / LO2 / LO3) it serves. References are to the course syllabus (3-Data Structure and Algorithm-CLC.pdf, INT1306_CLC).

Structure	Syllabus ref	LO	Built on	Built on top of it
Linear search	§2.2.2, p.3	LO2	array (unsorted)	—
Binary search	§2.2.3, p.3	LO2	sorted array	—
Interpolation search	§2.2.4, p.4	LO2	sorted array, uniform keys	—
Hashing & collisions	§2.2.5, p.4	LO2 + LO3	array (buckets) + linked list (chaining)	—

A.2 Motivation — one paragraph per structure

For each structure answer explicitly:

- What problem does it exist to solve? What was unacceptable before it?
- Which structures does it build on, and which are built on top of it?
- When is it the **wrong** choice? Name a concrete case where you would not use it.

A.3 Sources

- Course syllabus — *Data Structures & Algorithms (INT1306_CLC)*, PTIT,

3-Data Structure and Algorithm-CLC.pdf, accessed 2026-09-11. *

Part B — Algorithms & complexity

B.1 Operation table

Every operation in the topic is tabulated. The demo implements linear search and the hash table; binary and interpolation search are part of the topic (§2.2.3, §2.2.4) and are included for completeness. They are *not* used in the demo because they require a sorted array, and the demo dataset is deliberately random (see Part C). A number without a justification scores nothing.

Operation	Best	Average	Worst	Space	Why
Linear search by phone	O(1)	O(n)	O(n)	O(1)	No index — every element is compared until found; first match is O(1), a miss scans all n.
Binary search by phone	O(1)	O(log n)	O(log n)	O(1)	Halves the search space on each probe; requires a sorted array.
Interpolation search	O(1)	O(log log n)	O(n)	O(1)	Probes by value proportion; O(log log n) only for uniformly distributed keys, degenerates to O(n) on skewed data.
Hash insert	O(1)	O(1)	O(n)	O(1)	Hash to a bucket and append to its chain; expected O(1) at constant load factor, O(n) if every key collides into one chain.
Hash search	O(1)	O(1)	O(n)	O(1)	Hash to a bucket and walk the chain; expected O(1), worst case a single long chain.
Hash delete	O(1)	O(1)	O(n)	O(1)	Find the node in the chain, then unlink it; same bounds as search.
Rehash (load > 0.75)	O(n)	O(n)	O(n)	O(n)	Every key is reinserted into a new, doubled bucket array; amortised O(1) per insert.

B.2 Implementation notes (from the C++ source)

Moved verbatim from the README — the concrete choices behind the bounds above.

- CSV (src/phonebook.cpp:30-83): quoted "Do, Thanh Tuan" with ""

escapes, unquoted split on **last** comma, trim spaces, strip wrapping quotes on phone, drop trailing `\r`, skip blank lines, skip rows with empty name or non-digit phone.

- **Insert** (`src/phonebook.cpp:143-164`): reject empty, reject

non-digit phone (`Invalid phone number`), reject duplicate phone via hash (`Phone number is already exist`), else store `CapitalizeFirst(ToLower(name))` (ASCII-only) + index in hash table.

- **Delete** (`deleteContactByPhone`, option 8): O(n) — vector erase

shifts the tail + full hash rebuild so order is preserved. Missing phone prints `Phone number not found`.

- **Search**: `searchLinearByPhone` exact scan, `searchHashByPhone` chained

lookup, `searchLinearByName` case-insensitive scan via `toLowerCase`.

- **Hash** (`src/hashtable.cpp`): `hash = hash*31 + (unsigned char)(c-'0')`

with 64-bit wrap, `% numBuckets`; start `101` → `nextPrime`; rehash at `load > 0.75` to `nextPrime(2x)`; `isPrime` uses `6k±1`.

- **Timer** (`include/timer.hpp`): `timeIt ms, benchmark(work, 5)`

best-of-5, `printTaskDuration` prints `\nTook: Xms`.

- **Benchmark** (`src/main.cpp:11-63`, option 0): auto-loads CSV if empty,

picks `first / random / last` phones, times linear vs hash 5 runs each.

Part D — Cross-language comparison

D.1 What is being compared

Same handwritten algorithm, same input, same machine: a hand-rolled chained hash table and a hand-rolled linear scan over `contacts_100k.csv` (n=100000) in all five languages — no `dict`, `map`, or `HashMap` anywhere. This is a *language + runtime* comparison of equivalent code. It is deliberately **not** hand-written C++ vs Python's built-in `dict`: that would compare a teaching implementation against a production hash table (open addressing, optimized C) and would answer a different question.

D.2 The same program in five languages

Same prompts, same messages, same file split as C++. Differences are only what the runtime forces (64-bit wrap, stdin, timing API).

C++	Python	Go	JavaScript (Node)	Java
<code>src/main.cpp</code>	<code>python/phonebook/main.py</code>	<code>go/phonebook/main.go</code>	<code>javascript/phonebook/src/main.js</code>	<code>java/phonebook/src/com/phonebook</code>
<code>src/phonebook.cpp</code> + <code>include/phonebook.hpp</code>	<code>phonebook.py</code>	<code>phonebook.go</code>	<code>src/phonebook.js</code>	<code>PhoneBook.java</code>
<code>src/hashtable.cpp</code> + <code>include/hashtable.hpp</code>	<code>hashtable.py</code>	<code>hashtable.go</code>	<code>src/hashtable.js</code>	<code>HashTable.java</code>
<code>include/timer.hpp</code>	<code>timer.py</code>	<code>timer.go</code>	<code>src/timer.js</code>	<code>Timer.java</code>
<code>include/contact.hpp</code>	<code>contact.py</code>	<code>contact.go</code>	<code>src/contact.js</code>	<code>Contact.java</code>

D.3 Comparison table

Fill every cell. The last row — where you would use it — is what earns the marks.

Criterion	C++	Python	Java	JavaScript	Go
Built-in structure used	—	—	—	—	—
Manual implementation effort					
Runtime on your dataset					
Memory footprint					
Memory management					
Type safety					
Readability of the code					
Where you would use it					

D.4 Runtime growth — line chart

Guide: run across multiple dataset sizes (e.g. n = 10k, 50k, 100k, 500k, 1M) and plot the runtime growth curves of all 5 languages overlaid on one graph.

D.5 Memory profiling — bar chart

Guide: use language-specific profilers to capture exact peak memory: Valgrind/Massif (C++), tracemalloc (Python), VisualVM (Java), Chrome DevTools (Node/JS), pprof (Go).

D.6 Statistical fairness

Guide: run each test at least 5 times (warm-up runs do not count for Java/JS JIT compilers); report the average and the variance; explain why Java is slow on run 1 but fast on run 5 (JIT compilation).

D.7 Measurement method

Option `0` is unchanged: per target (`first/random/last`) it prints best-of-5:

```
[role index N phone P]
Linear best of 5: Xms. (index N)
Hash best of 5: Yms. (index N, position-independent)
```

Run the full matrix:

```
make run-benchmark # all 5 langs, every run → benchmark/results.csv (overwrite)
```

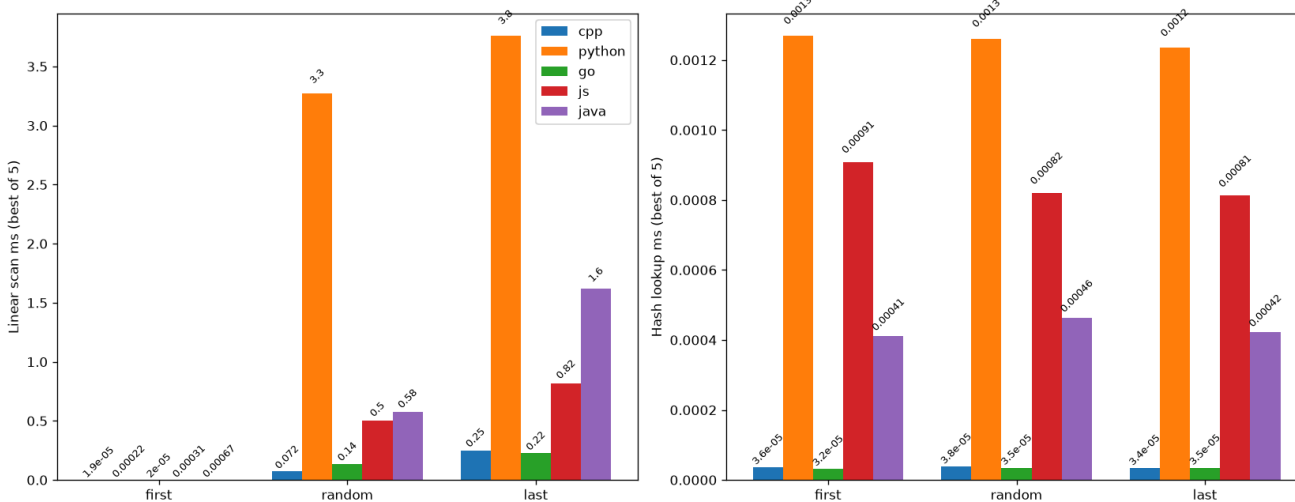
Each program also supports batch mode directly (same 30 rows each — compile the C++ demo first):

```
make # builds ./build/cpp/demo; other languages need no build step
./build/cpp/demo --benchmark-csv benchmark/results.csv data/contacts_100k.csv
python3 python/phonebook/main.py --benchmark-csv benchmark/results.csv --append data/contacts_100k.csv
(cd go/phonebook && go run . --benchmark-csv ../../benchmark/results.csv --append ../../data/contacts_100k.csv)
(cd javascript/phonebook && node src/main.js --benchmark-csv ../../benchmark/results.csv --append ../../data/contacts_100k.csv)
(cd java/phonebook && javac -d out src/com/phonebook/*.java && java -cp out com.phonebook.Main --benchmark-csv ../../benchmark/results.csv --append ../../data/contacts_100k.csv)
```

`benchmark/results.csv` columns: `language,dataset,n,case,algo,run,ms,timestamp,toolchain,target_index,phone` — 150 rows (5 langs × `first/random/last` × `linear/hash` × 5 runs). Batch mode seeds RNG with 42 so the `random` target is reproducible *within* a language (each language's RNG differs, so targets differ across languages — `target_index,phone` columns make that auditable). Option `0` stays unseeded.

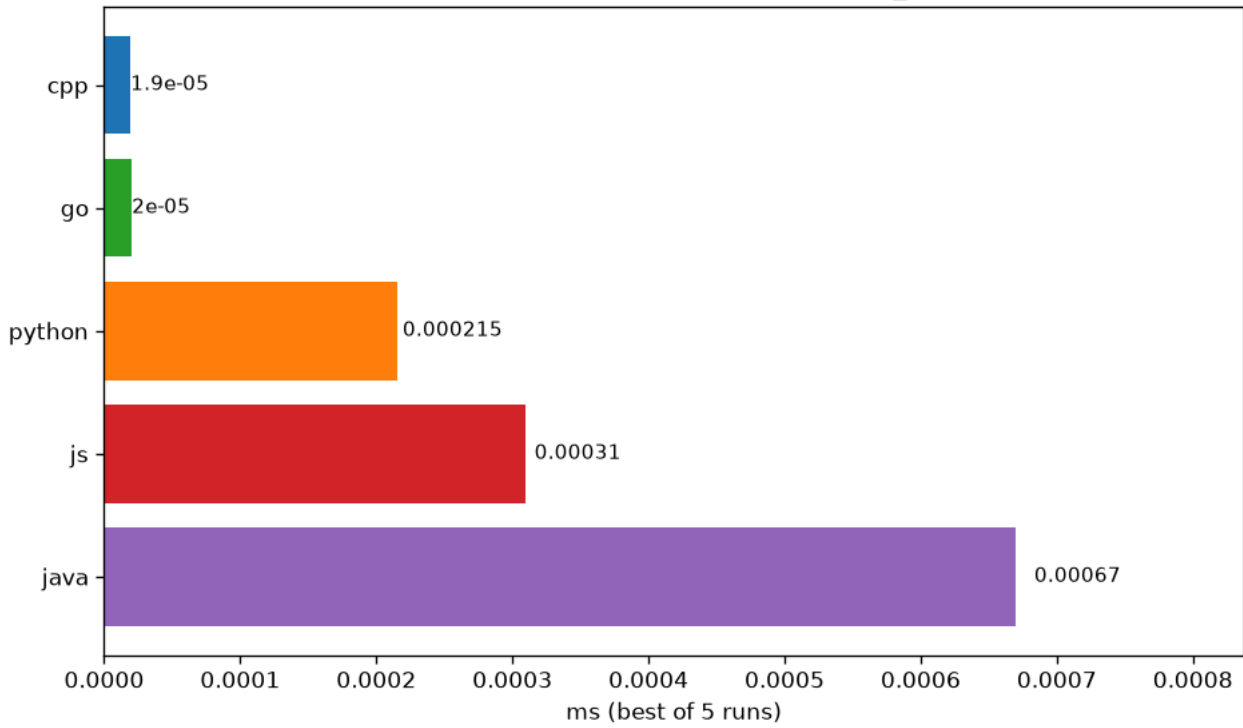
`make run-benchmark` also draws `benchmark/plot.png` at the end via `benchmark/plot.py` (two linear-scale panels — linear scan | hash lookup — best of 5 per cell with value labels; split axes so bar heights stay in true ratio and the `µs` hash bars aren't flattened by the `ms` linear bars; needs `matplotlib` — without it the plot step prints `plot skipped` and the CSV is still produced). The script parses every language's `ms` float format (C++ `setprecision(17)`, Python `repr`, Go `%g`, JS double, Java `Double.toString`) defensively: bad rows are skipped with a warning, never a crash.

Phone search: linear vs hash (`data/contacts_100k.csv`, `n=100000`)

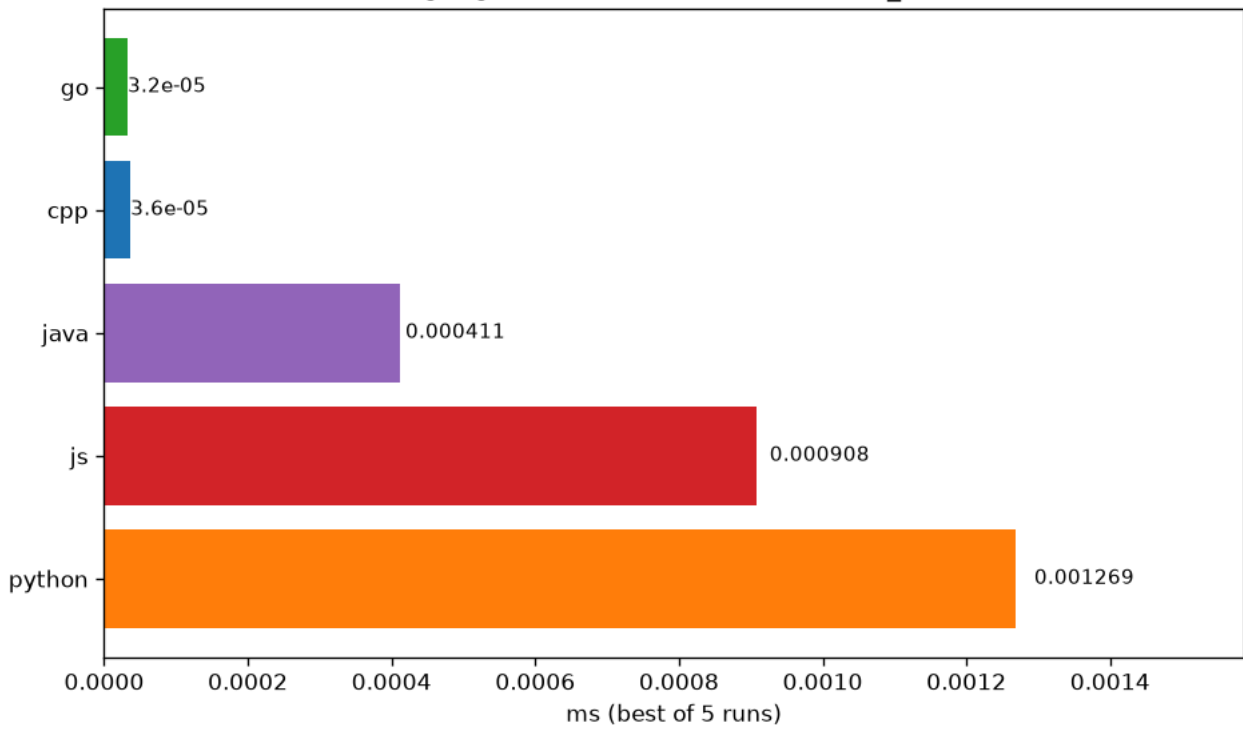


▼ Per-group charts (one image per case × algo, horizontal, fastest first)

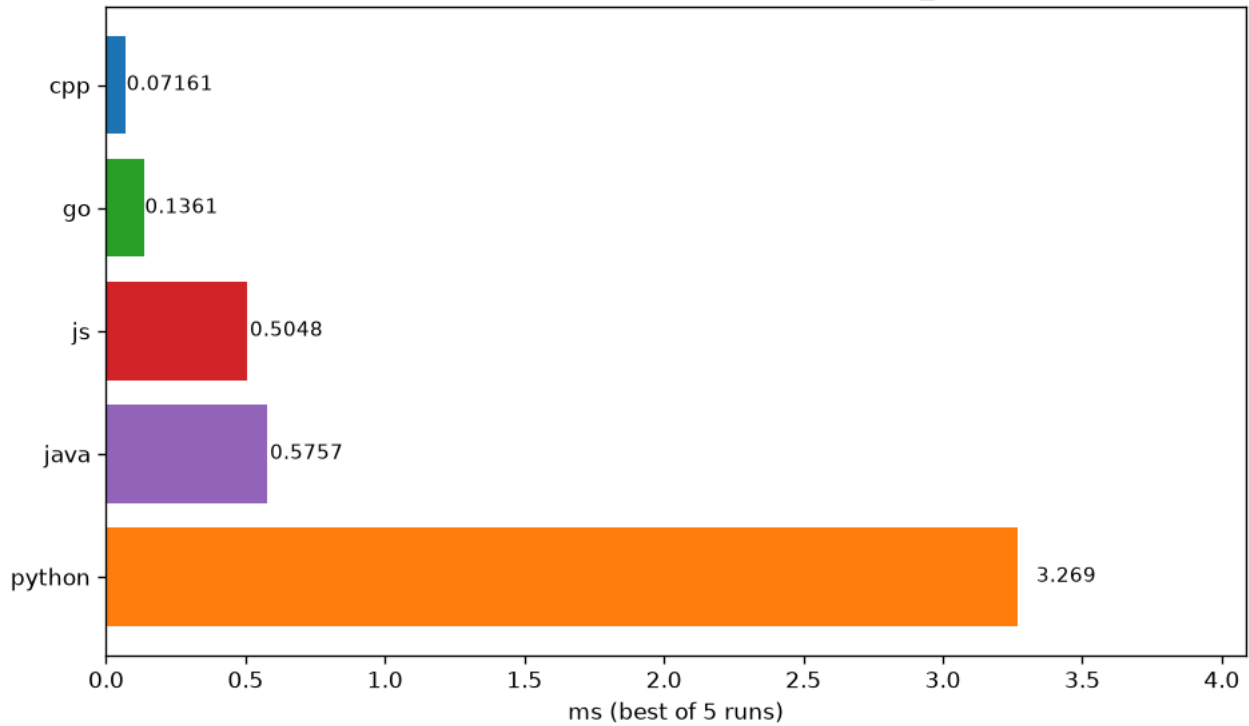
first-linear across languages, fastest first (data/contacts_100k.csv, n=100000)



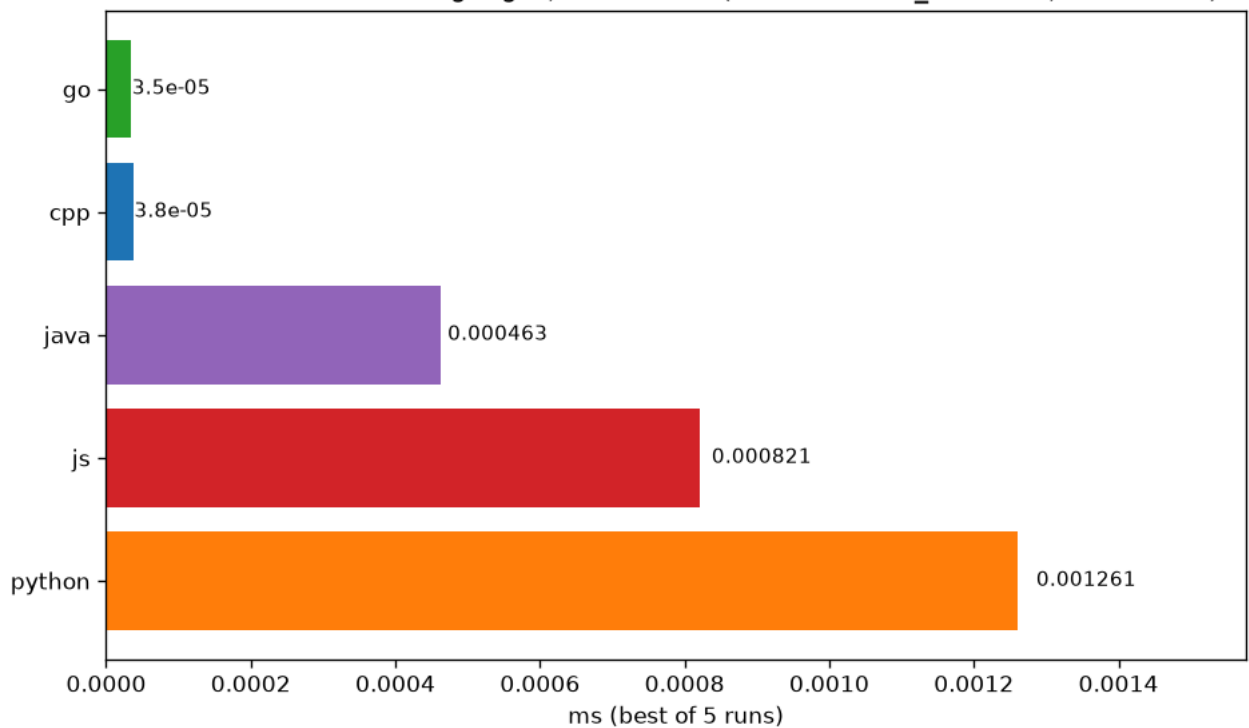
first-hash across languages, fastest first (data/contacts_100k.csv, n=100000)



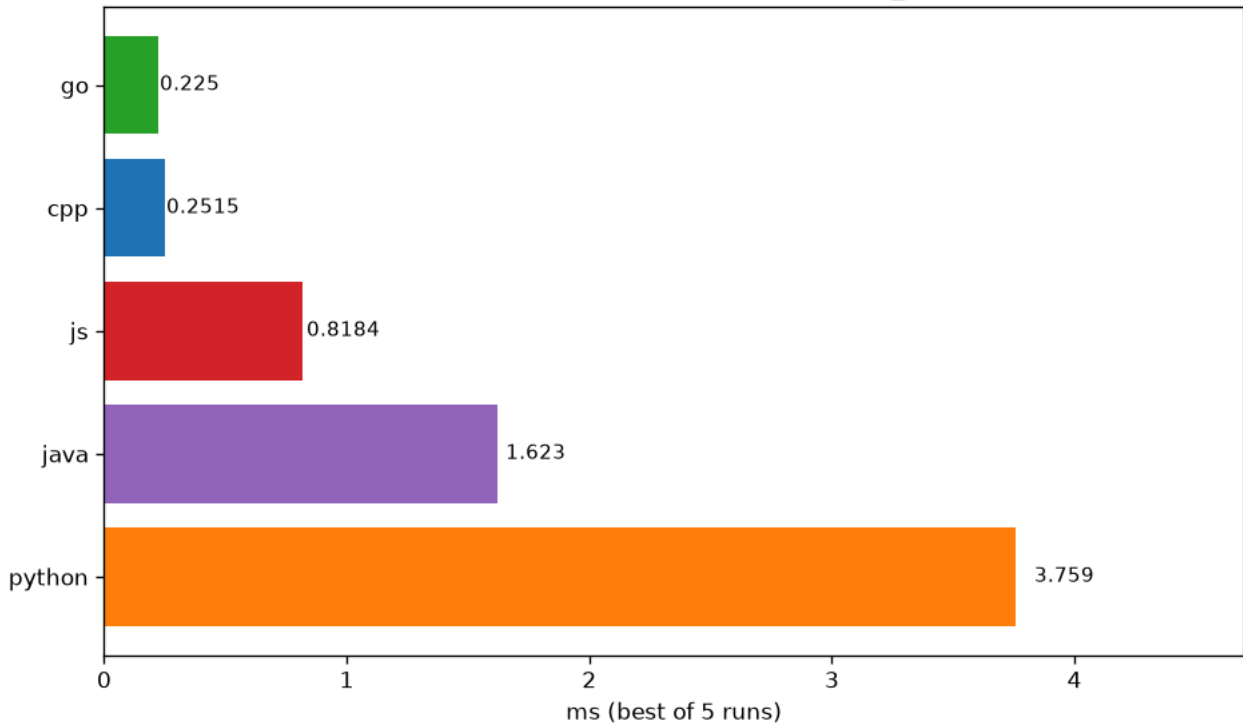
random-linear across languages, fastest first (data/contacts_100k.csv, n=100000)



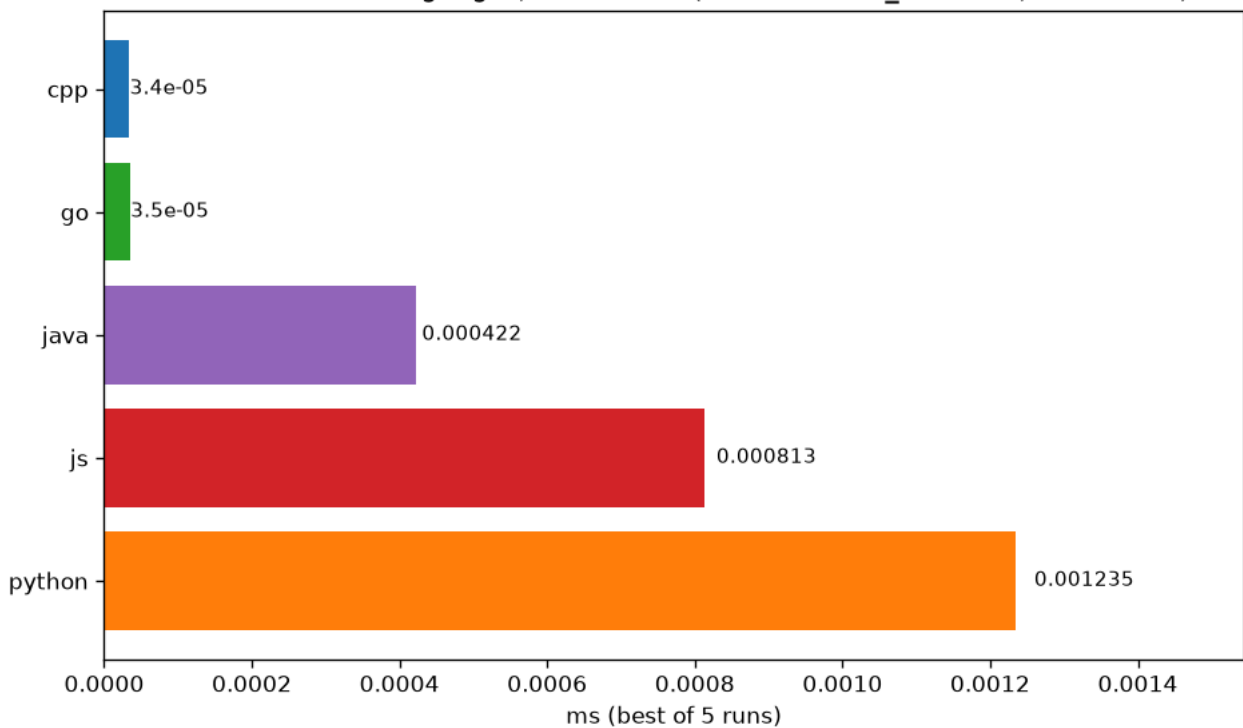
random-hash across languages, fastest first (data/contacts_100k.csv, n=100000)



last-linear across languages, fastest first (data/contacts_100k.csv, n=100000)



last-hash across languages, fastest first (data/contacts_100k.csv, n=100000)



D.8 Explaining the gap, not just observing it

"Python is slower" is an observation; the mechanism is the answer:

- **C++** (-O2) is the baseline: `vector<Contact>` stores structs

contiguously, short strings sit inside the object (SSO), and the scan compiles to a tight machine-code loop with no per-element overhead.

- **Python** pays per element: every `Contact`, `str`, and `int` is a

heap-boxed `PyObject` reached through a pointer, and each loop iteration runs CPython bytecode dispatch plus attribute-dict lookups. That is why `last-linear` is ~10× the others — 100k interpreter steps vs 100k machine instructions. The hash bars stay small because $O(1)$ steps mean the per-op tax applies ~once, not 100k times.

- **Go** is compiled with value-type slices, so its scan is a real

machine loop (bounds checks are nearly free here) — expect it closest to C++. Its gap is runtime checks + GC write barriers, not interpretation.

- **JavaScript (V8)** stores an array of heap objects (pointer chasing +

hidden-class checks per access) and must JIT-warm: best-of-5 lets the optimizing compiler settle, but run-to-run spread is JIT tier-up, not noise in your code.

- **Java** is the same story on the JVM: `ArrayList<Contact>` holds

references, the C2 compiler kicks in after ~10k loop iterations (one 100k scan trips it), and G1 may pause — startup is *not* measured, only the timed search, which flatters Java fairly since all languages get the same treatment.

Same position, except random. `first` (index 0) and `last` (index $n-1$) hit the identical phone in every language, so those bars are directly comparable. The `random` target is seeded (42) but each language has its own RNG, so the index differs per language — check `target_index` in the CSV before comparing `random-*` bars across languages.

What you may claim. The portable, algorithmic claim is the *trend within each language*: linear grows first→last, hash stays ~constant — it holds in all five, which is the DSA point. Cross-language magnitude differences are runtime properties; cite them only with the mechanism above, plus your machine spec (CPU/RAM, OS, toolchains, commit).

D.9 Recommendation

D.10 Measurement tables

Copy this table and fill from `benchmark/results.csv` + your machine spec:

Run	Dataset	CPU/RAM, OS	Toolchain	Commit	n	target	Linear runs (ms)	Hash runs (ms)
C++ ex	contacts_100k	i5-1135G7, 15 GiB, Omarchy	g++ 16.2.1, -O2	8b9be08	100000	first/random/last		
py ex	contacts_100k	i5-1135G7, 15 GiB, Omarchy	Python 3.14.7	8b9be08	100000	first/random/last		
go ex	contacts_100k	i5-1135G7, 15 GiB, Omarchy	go1.27.1	8b9be08	100000	first/random/last		
js ex	contacts_100k	i5-1135G7, 15 GiB, Omarchy	node v26.7.0	8b9be08	100000	first/random/last		
java ex	contacts_100k	i5-1135G7, 15 GiB, Omarchy	javac 27	8b9be08	100000	first/random/last		

D.11 Test conditions (machine & toolchain)

A measurement without its conditions is not a measurement. Same table as `README.md` — fill once, keep both in sync.

Component	Value
CPU	Intel Core i5-1135G7 @ 2.40 GHz (4 cores / 8 threads)
RAM	15 GiB
OS	Omarchy (Arch Linux), kernel 7.2.3-arch1-3
C++	g++ 16.2.1 (GCC), -O2 (see Makefile)
Python	Python 3.14.7
Go	go1.27.1 linux/amd64
Node	v26.7.0
Java	javac 27 (JDK 27)
Dataset	data/contacts_100k.csv (n=100000), seed 42
Commit	8b9be0826b14e05636361382a80029c5bef6b61b

Appendix

A.1 Test conditions

See Part D.11 — CPU/RAM, OS, compiler versions, optimisation flags, dataset and how it was generated.

A.2 Dataset

See `data/README.md`. Name,Phone per line, no header. Regenerate with `python3 data/generate.py --seed 42`. Use `contacts_50.csv` for demos, `100k/200k/1m` for scaling.

A.3 Port implementation notes

- **Python**: `time.perf_counter`, `random.randint`, 64-bit wrap via

& 0xFFFFFFFFFFFFFFFF, `(ord(c)-48) & 0xFF` to match `(unsigned char)(c-'0')`.

- **Go**: `uint64` wraps like C++ `size_t`, `time.Now`, `math/rand`,

`bufio.Reader` line input.

- **JavaScript (Node):** `perf_hooks.performance.now`, `Math.random`,

`readline/promises` async CLI, `BigInt` with `& 0xFFFFFFFFFFFFFFFFn` for hash wrap, `fs.readFileSync`.

- **Java:** `System.nanoTime()/1e6`, `java.util.Random`, `Scanner`

`hasNextInt/nextLine` mirroring `cin >> choice`, `long` wrap + `Long remainderUnsigned` for hash mod. `out/` is git-ignored.

A.4 Sources

A.5 Contribution statement

A.6 AI-use declaration